

Celebal technologies internship

Week-2 sql assignment

Name-Naman verma

Ques 1:output

```
--Q1. Write a query to display all columns and rows from the customer's table.  
SELECT * FROM customers;  
--this query help us see all of our data
```

id	customer_id	first_name	last_name	email	city	state	join_date	is_premium
101		Aarav	Sharma	aarav.s@em...	Mumbai	Maharashtra	2024-01-15	1
102		Priya	Patel	priya.p@em...	Ahmedabad	Gujarat	2024-02-20	0
103		Rohan	Gupta	rohan.g@em...	Delhi	Delhi	2024-03-10	1
104		Sneha	Reddy	sneha.r@em...	Hyderabad	Telangana	2024-04-05	0
105		Vikram	Singh	vikram.s@e...	Jaipur	Rajasthan	2024-05-12	1
106		Ananya	Iyer	ananya.i@e...	Chennai	Tamil Nadu	2024-06-18	0
107		Karan	Mehta	karan.m@e...	Pune	Maharashtra	2024-07-22	1

Ques2:output

```
--Q2. Retrieve only the first_name, last_name, and city of all customers  
SELECT first_name, last_name, city  
FROM customers;  
--this query retrieve onlt the given tablecustomers
```

id	first_name	last_name	city
	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Ques3:output

```
--Q3. List all unique categories available in the products table.  
SELECT DISTINCT category  
FROM products;
```

category
Clothing
Electronics
Home

Ques7:output

```
--Q7. Retrieve all orders with status = 'Delivered'.  
SELECT * FROM orders WHERE status = 'Delivered';
```

order_id	customer_id	order_date	status	total_amount
1001	101	2024-08-01	Delivered	4498
1002	102	2024-08-03	Delivered	799
1004	101	2024-08-10	Delivered	3499
1006	105	2024-08-15	Delivered	5898
1008	103	2024-08-20	Delivered	899
1010	108	2024-08-28	Delivered	1598

Ques8:output

```
--Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.  
SELECT * FROM products WHERE category = 'Electronics' AND unit_price > 2000;
```

product_id	product_name	category	brand	unit_price	stock_qty
203	Smart Watch	Electronics	Noise	2999	150
205	Bluetooth Speaker	Electronics	JBL	3499	200

Ques9:output

```
--Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.  
SELECT * FROM customers WHERE join_date BETWEEN '2024-01-01' AND '2024-12-31' AND state = 'Maharashtra';
```

i	custome...	first_name	last_name	email	city	state	join_date	is_premium
101		Aarav	Sharma	aarav.s@em...	Mumbai	Maharashtra	2024-01-15	1
107		Karan	Mehta	karan.m@e...	Pune	Maharashtra	2024-07-22	1

Ques10:output

```
--Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled
SELECT * FROM orders WHERE order_date BETWEEN '2024-08-10' AND '2024-08-25' AND status <> 'Cancelled';
```

i	order_id	customer_id	order_date	status	total_amount
1004		101	2024-08-10	Delivered	3499
1006		105	2024-08-15	Delivered	5898
1007		106	2024-08-18	Pending	1299
1008		103	2024-08-20	Delivered	899
1009		107	2024-08-25	Shipped	6098

Ques11:output

```
--Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that
--filters orders by order_date? Write a sample query that would benefit from this index.
--idx order dates works like a index for the table like i help find the exact location of the date rather than going
SELECT * FROM orders WHERE order_date = '2024-08-15';
```

i	order_id	customer_id	order_date	status	total_amount
1006		105	2024-08-15	Delivered	5898

Ques12:output

```
SELECT * FROM customers WHERE join_date >= '2024-01-01' AND join_date <= '2024-12-31';
```

i	custome...	first_name	last_name	email	city	state	join_date	is_premium
101		Aarav	Sharma	aarav.s@em...	Mumbai	Maharashtra	2024-01-15	1
102		Priya	Patel	priya.p@em...	Ahmedabad	Gujarat	2024-02-20	0
103		Rohan	Gupta	rohan.g@em...	Delhi	Delhi	2024-03-10	1
104		Sneha	Reddy	sneha.r@em...	Hyderabad	Telangana	2024-04-05	0
105		Vikram	Singh	vikram.s@e...	Jaipur	Rajasthan	2024-05-12	1
106		Ananya	Iyer	ananya.i@e...	Chennai	Tamil Nadu	2024-06-18	0
107		Karan	Mehta	karan.m@e...	Pune	Maharashtra	2024-07-22	1
108		Divya	Nair	divya.n@em...	Kochi	Kerala	2024-08-30	0

Ques14:output

```
--Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.
SELECT SUM(total_amount) AS total_delivered_revenue FROM orders WHERE status = 'Delivered';
```

total_delivered_revenue

17191

Ques15:output

```
--Q15. Calculate the average unit_price of products in each category.
SELECT category, AVG(unit_price) AS avg_unit_price FROM products GROUP BY category;
--CLOTHING holds the highest avg_unit price
```

category	avg_unit_price
Clothing	2699
Electronics	2224
Home	949

Ques16:output

```
--Q16. For each order status,
--find the count of orders and the total revenue. Sort the result by total revenue in descending order.
SELECT status, COUNT(*) AS order_count, SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;
--While Delivered revenue looks good, our Shipped pipeline holds a massive ₹13,596.00 in transit.
--Moving these orders to "Delivered" quickly is crucial to keep our cash flow healthy.
```

status	order_count	total_revenue
Delivered	6	17191
Shipped	2	13596
Cancelled	1	2999
Pending	2	2897

Ques17:output

```
--Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.
SELECT category, MAX(unit_price) AS most_expensive, MIN(unit_price) AS cheapest
FROM products
GROUP BY category;
--this table shows the most expensive and cheapest item of each category
```

category	most_expensive	cheapest
Clothing	4599	799
Electronics	3499	899
Home	1299	599

Ques 18:

```
--Q18. List all product categories where the average unit_price is greater than ₹2000.
SELECT category, AVG(unit_price) AS avg_unit_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;
--This confirms that Clothing and Electronics are our primary high-value drivers,
--while Home stays below our ₹2,000 baseline.
```

category	avg_unit_price
Clothing	2699
Electronics	2224

Ques 19:

```
--Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name.
--Show: order_id, order_date, first_name, last_name, total_amount.
SELECT o.order_id, o.order_date, c.first_name, c.last_name, o.total_amount
FROM orders o
INNER JOIN customers c ON o.customer_id = c.customer_id;
--an INNER JOIN allows us to instantly see exactly
--who placed each order without manually matching IDs between different tables.
```

order_id	order_date	first_name	last_name	total_amount
1001	2024-08-01	Aarav	Sharma	4498
1002	2024-08-03	Priya	Patel	799
1003	2024-08-05	Rohan	Gupta	7498
1004	2024-08-10	Aarav	Sharma	3499
1005	2024-08-12	Sneha	Reddy	2999
1006	2024-08-15	Vikram	Singh	5898
1007	2024-08-18	Ananya	Iyer	1299
1008	2024-08-20	Rohan	Gupta	899

Ques 20:

```
--Q20. Using a LEFT JOIN, list ALL customers and their orders (if any).
--Customers with no orders should still appear with NULL values for order columns.
SELECT c.customer_id, c.first_name, c.last_name, o.order_id, o.status, o.total_amount
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id;
```

customer_id	first_name	last_name	order_id	status	total_amount
101	Aarav	Sharma	1001	Delivered	4498
101	Aarav	Sharma	1004	Delivered	3499
102	Priya	Patel	1002	Delivered	799
102	Priya	Patel	1011	Pending	1598
103	Rohan	Gupta	1003	Shipped	7498
103	Rohan	Gupta	1008	Delivered	899
104	Sneha	Reddy	1005	Cancelled	2999
105	Vikram	Singh	1006	Delivered	5898

Ques 21:

```
--Q21. Write a query using JOINS across three tables (orders → order_items → products) to show:
--order_id, product_name, quantity, unit_price, and discount_pct for each order item.
SELECT oi.order_id, p.product_name, oi.quantity, oi.unit_price, oi.discount_pct
FROM order_items oi
INNER JOIN orders o ON oi.order_id = o.order_id
INNER JOIN products p ON oi.product_id = p.product_id;
```

order_id	product_name	quantity	unit_price	discount_pct
1001	Wireless Earbuds	2	1499	0
1001	Laptop Stand	1	899	10
1002	Cotton T-Shirt	1	799	0
1003	Smart Watch	1	2999	0
1003	Running Shoes	1	4599	5
1004	Bluetooth Speaker	1	3499	0
1005	Smart Watch	1	2999	0
1006	Wireless Earbuds	1	1499	10

Ques 24:

```
--Q24. Write a query using CASE to classify products into price tiers:
-- • 'Budget'    → unit_price < 1000
-- • 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
-- • 'Premium'   → unit_price > 3000
--Display: product_name, unit_price, price_tier.
```

```
SELECT product_name, unit_price,
       CASE
         WHEN unit_price < 1000 THEN 'Budget'
         WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'
         ELSE 'Premium'
       END AS price_tier
FROM products;
```

product_name	unit_price	price_tier
Wireless Earbuds	1499	Mid-Range
Cotton T-Shirt	799	Budget
Smart Watch	2999	Mid-Range
Running Shoes	4599	Premium
Bluetooth Speaker	3499	Premium
Bedsheet Set	1299	Mid-Range
Laptop Stand	899	Budget
Cushion Covers (Set)	599	Budget

Ques 25:

```
--Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered'
--Display the result in a single row.
SELECT
    SUM(CASE WHEN status = 'Delivered' THEN 1 ELSE 0 END) AS delivered_count,
    SUM(CASE WHEN status <> 'Delivered' THEN 1 ELSE 0 END) AS not_delivered_count
FROM orders;
```

delivered_count	not_delivered_count
6	5

Ques 27:

```
-Q27. Write a SQL transaction that does the following atomically:
--1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
--2. Insert two order items for that order
--3. Update the stock_qty of the purchased products
--4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.
-Write the complete BEGIN...COMMIT/ROLLBACK block.
```